

EnviroGuard

Product Requirements Document — v1.0

Scaffold edition — starting point for the software team, not a finished product

Status	Scaffold — placeholder UI, fake data, real structure
Platform	iOS + Android (Expo) — tablet kiosk + phone
Backend	One EC2 server (FastAPI) + DynamoDB + Anthropic API
Routing	React Navigation — bottom tab + stack navigators
Language	TypeScript
AI	Anthropic API — Claude LLM + Claude Vision

1 — What this document is

This PRD describes EnviroGuard as a scaffold — a structured starting point for the software team, not a finished product. The app should be built with all five tabs present, all buttons functional, and all screens navigable, but with placeholder data, temporary UI, and clear signposting of what still needs to be connected or finished. Think of it as a skeleton with muscles but no skin.

How the scaffold should work

- Every screen exists and is reachable — no dead tabs, no placeholder 'coming soon' pages
- All buttons and interactions work — they trigger the right navigation or show a loading state
- Data is hardcoded fake JSON that matches the real API contract shape exactly
- Every file has comments explaining what it does, what needs to be connected, and how
- The README inside the app folder lists every TODO, every env variable needed, and every endpoint to wire up
- UI components are clearly marked as changeable — layout and colors are provisional, structure is real
- When the backend is ready, swapping fake data for real data should be a one-file change per screen

README specification

The app repository must contain a README.md at the root with the following sections. This README is written for whoever picks up the project next — human developer or AI tool.

Project overview

One paragraph describing EnviroGuard — what it is, what it does, what the three data sources are.

Architecture diagram (text)

ASCII or text diagram showing: Phone/Tablet App → EC2 FastAPI server → DynamoDB. Plus EC2 → Anthropic API for Claude calls.

Setup instructions

Step-by-step: install Expo CLI, clone repo, copy .env.example to .env, fill in variables, run npx expo start.

Environment variables

Table of every env var: name, what it is, where to get it. EC2_BASE_URL, ANTHROPIC_API_KEY, AWS_REGION, DYNAMODB_TABLE_.*.

What is real vs placeholder

Explicit list: fake data files, hardcoded coordinates, placeholder photos, mock user profile.

What still needs to be built

Numbered TODO list — every unfinished feature, every unconnected endpoint, every screen that needs real data.

API endpoints reference

Table of every endpoint the app calls, the expected request shape, and the expected response shape.

How to add a new screen

Brief instructions for future developers or AI tools on the file structure and navigation pattern to follow.

Claude API usage

List of every Claude call in the app, which file it lives in, and what prompt template it uses.

For AI tools

A section explicitly addressed to AI coding assistants: 'If you are an AI tool reading this, here is the context you need...' followed by the project summary, stack, conventions, and current state.

2 — Product overview

EnviroGuard is a neighborhood environmental intelligence app built around physical station kiosks. Each station sits next to a public trash can and contains three sensors: a microphone monitoring noise levels, an ultrasonic sensor measuring bin fill percentage, and a camera watching for litter and hazards. The same Expo app runs on the station tablet (kiosk mode) and on any smartphone. Sensor data flows to a FastAPI server on EC2, which stores it in DynamoDB and serves it to the app. Claude provides AI: explaining risk, analyzing photos, and drafting advocacy letters.

Three data sources

Source	Sensor	What it measures	Cloud processing
Noise	Station mic (USB)	Ambient dB every 5 sec	Prophet model — predicts dB by hour
Trash	HC-SR04 ultrasonic + Pi Camera	Bin fill % + litter severity	YOLOv8-nano litter CV + Prophet fill forecast
Hazards	Pi Camera + phone camera	Flooding, dumping, spills	MobileNet flag + Claude Vision full analysis

3 — Backend architecture

The backend is intentionally simple — one EC2 server does everything. The app talks to one base URL for all data. There is no API Gateway, no Lambda, no complex AWS orchestration. Just three things:

Service	What it is	What it does
EC2 (one server)	A virtual machine running FastAPI (Python)	Handles all API routes, talks to DynamoDB, runs ML models, proxies Claude calls
DynamoDB	AWS NoSQL database	Stores all sensor readings, reports, users, posts, events, and alerts
Anthropic API	External AI service	Claude LLM for summaries and letters, Claude Vision for photo analysis

How data flows

The app makes HTTP requests to the EC2 server. The EC2 server reads or writes DynamoDB and returns a response. For ML predictions, the same EC2 server runs the FastAPI app and the Python ML models — no separate server needed. For Claude calls, EC2 either proxies them server-side or the app calls the Anthropic API directly with the key stored in the app's environment. The Pi station posts sensor data directly to the EC2 server via HTTP POST.

EC2 API endpoints — full reference

All endpoints are on the EC2 FastAPI server. Base URL stored in EC2_BASE_URL environment variable. In development, fake data is returned from local JSON files at the same paths.

Meth od	Path	Description	Returns
GET	/stations	List all stations with location	Array of station objects
GET	/readings/{station_id}	Latest sensor readings for a station	noise_db, bin_fill_pct, timestamp
GET	/readings/{station_id}/history	Historical readings for Prophet training	Array of timestamped readings
GET	/reports?lat=&lng;=&radius;=	Reports near a location	Array of report objects
POST	/reports	Submit a new hazard report	Created report object

GET	/predict-noise/{station_id}	24h noise forecast from Prophet	Array of {hour, db, lower, upper}
GET	/predict-fill/{station_id}	48h bin fill forecast + overflow time	Array of {hour, fill_pct} + overflow_at
POST	/detect-litter	Run YOLOv8 on a camera frame	{severity, label, confidence}
POST	/analyze-photo	Claude Vision on a user photo	{type, severity, description, agency}
GET	/users/{user_id}	Fetch user profile	Health conditions, thresholds, locations
PUT	/users/{user_id}	Update user profile	Updated user object
GET	/posts?lat=&lng;=	Community feed near location	Array of post objects
POST	/posts	Create a community post or event	Created post object

DynamoDB tables

Table name	Partition key	Sort key	What it stores
sensor_readings	station_id (String)	timestamp (String)	All mic dB and ultrasonic fill readings
hazard_reports	report_id (String)	—	Photo reports, Claude Vision analysis, status
users	user_id (String)	—	Health profile, thresholds, saved locations
posts	post_id (String)	—	Community feed posts and events
alerts	alert_id (String)	—	Alert history per user

Environment variables

These go in a .env file at the app root. The scaffold includes a .env.example with placeholder values. Never commit real values to the repo.

Variable	What it is	Placeholder value
EC2_BASE_URL	Base URL of the EC2 FastAPI server	http://localhost:8000 (dev)

ANTHROPIC_API_KEY	Anthropic API key for Claude calls	sk-ant-REPLACE_ME
AWS_REGION	AWS region where DynamoDB lives	us-east-1
DYNAMODB_TABLE_READINGS	Sensor readings table name	sensor_readings
DYNAMODB_TABLE_REPORTS	Hazard reports table name	hazard_reports
DYNAMODB_TABLE_USERS	Users table name	users
DYNAMODB_TABLE_POSTS	Posts and events table name	posts
KIOSK_MODE	Set to 'true' on tablet build	false

4 — Design language

■ **SCAFFOLD NOTE:** UI is provisional and changeable. The scaffold uses these colors and spacing as defaults. The team should feel free to adjust any visual — the structure and navigation are what matter, not the polish.

Color system

Role	Hex	Used for
Primary	#0F6E56	CTAs, active tab, map heatmap anchor
Primary mid	#1D9E75	Secondary actions, sensor indicators
Primary light	#E1F5EE	Card backgrounds, selected rows
Danger	#D85A30	High-risk alerts, severity 4–5 badges
Warning	#EF9F27	Medium risk, severity 3 badges
Safe	#639922	Low risk, severity 1–2 badges
Text primary	#2C2C2A	All body copy and labels
Text secondary	#888780	Metadata, timestamps, helper text
Border	#D3D1C7	Card outlines, dividers, input borders
Background	#F1EFE8	App background

Typography + spacing

- Font: system default — SF Pro on iOS, Roboto on Android. No custom fonts.
- Screen titles: 20–22px bold. Body: 14–15px regular. Metadata: 12px regular.
- Data values (dB, fill %, scores): fontVariant tabular-nums to prevent layout shift.
- Base spacing unit: 8px. All margins and padding are multiples of 8.
- Screen horizontal padding: 16px. Card internal padding: 12–16px. Section gaps: 24px.

Key components

- Risk score pill: rounded rectangle, color by severity, number bold white, fixed 44px width.
- Severity badge: number 1–5 in a 24px colored circle. 1–2 green, 3 amber, 4–5 red.

- Report card: white, 12px radius, 1px border (#D3D1C7), 12px padding, severity badge top-right.
- Loading states: skeleton screen for lists, spinner for Claude API calls.
- Empty states: centered text description — never a blank screen.
- Layer toggle chips: horizontal scroll row, teal when active, outlined when inactive.
- Kiosk mode: all touch targets minimum 56px, no text under 16px, bottom tabs hidden.

5 — Navigation structure

Navigator	Screens	Notes
Auth Stack	LoginScreen, OnboardingScreen (3 steps)	Shown when no token in SecureStore
Tab: Map	MapScreen → ZoneDetailScreen	ZoneDetail is bottom sheet
Tab: Health	HealthScreen → AlertDetailScreen	AlertDetail shows source data
Tab: Report	ReportFeedScreen → ReportDetailScreen, SubmitReportScreen	Submit: camera → Claude Vision → confirm
Tab: Community	CommunityScreen → EventDetailScreen, LetterScreen	LetterScreen: streaming Claude output
Tab: Profile	ProfileScreen → EditHealthScreen, ThresholdsScreen	Simple settings stack

Kiosk mode

- Boot directly to MapScreen — no auth required for view-only access
- Bottom tab bar hidden, navigation locked to map
- expo-keep-awake prevents device sleep
- 5-tap hidden gesture on map → PIN screen → exit kiosk
- Detected via KIOSK_MODE environment variable set at build time

6 — Tab specifications

■ SCAFFOLD NOTE: Each tab card below lists: what the tab does, what the placeholder UI should look like (temporary, changeable), what AWS endpoints it needs to connect to, and what TODOs remain. All of these TODOs should appear as comments in the code and as checklist items in the README.

Tab 1 Map

Live risk map — unified view + 3 toggleable layers

What it does

Home screen. Full-bleed map showing a unified risk heatmap. Three layer chips at the top toggle Noise, Trash, and Hazards independently. Tapping a zone opens a bottom sheet with a Claude risk explanation. Alert push notifications fire from this tab.

Placeholder UI (changeable)

- MapView placeholder: grey rectangle with 'Map goes here — swap for Mapbox GL' label
- Fake heatmap overlay: 3 colored circles at hardcoded NYC coordinates
- Layer chips: tappable, toggle active state visually, no real data filter yet
- Bottom sheet: appears on map tap, shows hardcoded fake risk text + loading spinner for Claude
- Markers: 3 fake report markers at hardcoded coordinates

AWS endpoints needed

- GET /stations — load station locations for markers
- GET /readings/{station_id} — current sensor values per zone
- GET /reports?lat=&lng;=&radius;= — nearby report markers

TODOs to connect + finish

- Replace placeholder MapView with Mapbox GL and real heatmap data
- Connect layer chips to real sensor data per layer
- Replace hardcoded markers with live DynamoDB report feed
- Wire Claude zone summary to real readings from /readings/{id}
- Implement push notification registration and threshold check

Tab 2 Health & Alerts

Forecast dashboard + personalized risk feed

What it does

24-hour forecast timeline at top — scrub to see predicted noise dB, bin fill %, and hazard risk by hour. Below: personal risk feed where Claude interprets the forecast through the user's health profile. Alert history at the bottom.

Placeholder UI (changeable)

- Timeline: horizontal ScrollView with 24 hardcoded hourly bars, values from fake JSON
- Metric selector: toggle buttons for Noise / Fill / Hazard — switches which fake data shows
- Claude risk feed: hardcoded string 'Your personalized briefing will appear here' + button that calls Claude with fake data
- Alert history: FlatList of 5 fake alert objects with timestamps

AWS endpoints needed

- GET /predict-noise/{station_id} – Prophet noise forecast
- GET /predict-fill/{station_id} – Prophet fill forecast + overflow time
- GET /users/{user_id} – health profile to pass to Claude
- GET /alerts – alert history for current user

TODOs to connect + finish

- Replace fake timeline data with real EC2 prediction endpoint responses
- Connect health profile from DynamoDB into Claude briefing prompt
- Wire alert history to real alerts table in DynamoDB
- Add confidence interval shading to timeline bars
- Implement proactive alert logic — fire before predicted threshold breach

Tab 3 Report

Submit reports + browse what's been flagged nearby

What it does

Opens on a scrollable feed of recent reports near the user. Floating action button opens the submission flow: camera → Claude Vision analysis → confirm → post. Each card shows type, severity, distance, status.

Placeholder UI (changeable)

- Feed: `FlatList` of 5 fake report objects — hardcoded type, severity, distance, timestamp
- Report card: layout complete with severity badge, type icon, distance chip, status pill
- FAB: navigates to `SubmitReportScreen` — button wired, navigation works
- `SubmitReportScreen`: `expo-image-picker` opens camera, selected photo shows in preview, 'Analyze' button calls Claude Vision with real image
- Claude Vision result: shows parsed JSON fields in a card — type, severity, description, agency
- Confirm button: logs to console, shows success toast — does not POST to DynamoDB yet

AWS endpoints needed

- `GET /reports?lat=&lng;=&radius;=` — real report feed
- `POST /reports` — submit confirmed report to DynamoDB
- `POST /analyze-photo` — Claude Vision analysis (can call Anthropic direct or via EC2)

TODOs to connect + finish

- Replace fake report feed with real `GET /reports` endpoint
- Wire confirm button to `POST /reports` with full report payload
- Upload photo to EC2 or S3 before Claude Vision call
- Add filter UI — by source (station/phone) and severity
- Connect station auto-reports — poll for new reports every 30 seconds

Tab 4 Community

Events, petitions, and advocacy letter generator

What it does

Neighborhood-scoped feed of posts and events.
Letter generator is the primary feature: Claude drafts a formal advocacy letter using real sensor readings as evidence, streamed into an editable field.

Placeholder UI (changeable)

- Feed: FlatList of 3 fake post objects and 2 fake events — hardcoded
- Post card: text, fake location chip, upvote count (tappable, increments locally)
- Event card: title, date, fake RSVP count, 'Add to Calendar' button (expo-calendar wired)
- Create Event button: navigates to form, form submits locally — not yet POSTed
- 'Generate Letter' button: navigates to LetterScreen, Claude called with hardcoded fake sensor readings, streaming works
- LetterScreen: editable TextInput pre-filled with streaming Claude output, 'Export PDF' button wired via expo-print

AWS endpoints needed

- GET /posts?lat=&lng;= – real community feed
- POST /posts – create post or event in DynamoDB
- GET /readings/{station_id} – real sensor data to pass to Claude letter prompt

TODOs to connect + finish

- Replace fake feed with real GET /posts endpoint
- Wire event and post creation to POST /posts
- Replace hardcoded sensor readings in letter prompt with real readings from /readings/{id}
- Add council district geocoding to auto-fill official name in letter
- Implement co-sign — POST an existing letter_id to associate another user

Tab 5 Profile

Health conditions, alert thresholds, saved locations

What it does

Minimal settings screen personalizing every other tab. Health conditions feed Claude advice in Health tab. Thresholds control push alerts. Saved locations set map default. Synced across phone and tablet.

Placeholder UI (changeable)

- Health conditions: multi-select list — asthma, COPD, sleep disorder, mobility, age group — selection stored in local state
- Alert thresholds: 3 sliders (noise dB, bin fill %, hazard radius) — values in local state
- Saved locations: 3 rows (Home, Work, School) with fake addresses, Edit button navigates to map pin screen (map pin screen is a placeholder)
- Account section: fake email displayed, Change Password and Delete Account buttons navigate to placeholder screens
- Save button: logs profile to console, shows success toast — not yet POSTed to DynamoDB

AWS endpoints needed

- GET /users/{user_id} — load existing profile on mount
- PUT /users/{user_id} — save profile changes to DynamoDB

TODOs to connect + finish

- Load profile from GET /users/{id} on screen mount
- Wire Save button to PUT /users/{id} with full profile payload
- Implement map pin picker screen for saved locations
- Sync profile to kiosk tablet on login — same DynamoDB user record
- Connect health conditions to Claude prompt in Health tab
- Connect thresholds to push alert trigger logic in Health tab

7 — Claude API usage

■ **SCAFFOLD NOTE:** In the scaffold, Claude calls should work with real API calls using fake/hardcoded input data. The prompt templates below are the ones to implement. When real sensor data is connected, only the input data changes — the prompt structure stays the same.

1 — Map zone summary (Tab 1, ZoneDetailScreen)

Prompt template:

```
system: "You are an environmental health advisor. Be direct and plain. 2-3 sentences max." user:
"Zone: {location_name}. Noise: {noise_db}dB. Bin fill: {fill_pct}%. Litter severity: {litter}/5.
Explain the risk to a resident in plain English."
```

Output: Plain text, 2–3 sentences. No JSON. Called when bottom sheet opens.

2 — Health briefing (Tab 2, HealthScreen)

Prompt template:

```
system: "You are a personal health advisor for environmental risk. Be specific, not generic." user:
"Health profile: {conditions}. Forecast: noise peaks at {peak_db}dB at {peak_hour}, bin predicted
to overflow at {overflow_time}. Give a personalized briefing."
```

Output: Plain text, 3–5 sentences + 2 action bullets. Called on screen mount.

3 — Photo analysis (Tab 3, SubmitReportScreen) — Vision call

Prompt template:

```
system: "Analyze this photo for environmental hazards. Respond ONLY in valid JSON, no other text."
user: [base64 image] + "Return: { type, severity (1-5), description, health_impact, agency, action
}"
```

Output: JSON only. Parse immediately. severity drives badge color on report card.

4 — Advocacy letter (Tab 4, LetterScreen) — streaming

Prompt template:

```
system: "Draft a formal civic advocacy letter. Tone: serious, factual, respectful. Use sensor data
as evidence." user: "Station {station_id} at {address}. Noise avg {avg_db}dB for {days} days (WHO
limit: 70dB). Bin at {fill_pct}%+ for {overflow_days} of {total_days} days. Official:
{official_name}, {district}. Resident: {user_name}. Draft the letter."
```

Output: Streamed plain text. Full letter with salutation and sign-off. Display token by token.

8 — Master TODO list

Everything in this list should appear as a comment in the relevant file AND as a checklist item in the README. This is the source of truth for what the scaffold does not yet do.

Infrastructure

- | | |
|---|--|
| ■ | Create EC2 instance (t3.small minimum), install Python, clone FastAPI repo, start server |
| ■ | Create DynamoDB tables: sensor_readings, hazard_reports, users, posts, alerts |
| ■ | Fill .env with real EC2_BASE_URL, ANTHROPIC_API_KEY, AWS_REGION, table names |
| ■ | Test all 13 API endpoints return real data before connecting app |
| ■ | Set up EC2 security group: open port 8000 (or 443 with nginx) to app requests |

App — data connections

- | | |
|---|---|
| ■ | Replace all fake JSON imports with real API calls to EC2_BASE_URL |
| ■ | Implement auth: user login → receive JWT → store in expo-secure-store |
| ■ | Load user profile from GET /users/{id} on app start, store in global context |
| ■ | Tab 1: connect map markers and heatmap to real /readings and /reports endpoints |
| ■ | Tab 2: connect timeline to real /predict-noise and /predict-fill endpoints |
| ■ | Tab 3: wire report submission to POST /reports, upload photo to EC2 or S3 |
| ■ | Tab 4: replace hardcoded letter sensor readings with live /readings data |
| ■ | Tab 5: wire Save button to PUT /users/{id} |

App — features not yet built

- | | |
|---|---|
| ■ | Mapbox GL integration — replace placeholder MapView with real heatmap |
| ■ | Push notifications — register Expo push token, store in users table, wire SNS |
| ■ | Council district geocoding — auto-fill official name in letter generator |
| ■ | Map pin picker screen for saved locations (Profile tab) |
| ■ | Station auto-reports — poll GET /reports every 30 seconds, update map markers |

- | | |
|---|--|
| ■ | Report feed filtering — by source, severity, and date range |
| ■ | Community feed real-time updates — poll or WebSocket |
| ■ | Kiosk PIN screen — 5-tap gesture + PIN entry to exit kiosk mode |
| ■ | Onboarding flow — 3-step health profile on first launch |
| ■ | Offline mode — cache last known data in AsyncStorage, show stale indicator |

EnviroGuard

PRD v1.0 — Scaffold Edition

Expo · React Navigation · TypeScript · EC2 FastAPI · DynamoDB · Anthropic API · YOLOv8 · Prophet